

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Organización de Computadores
Parcial 1 — Documento de Estrategia

ALU de 32 bits y ALU de tres entradas sobre Nand2Tetris

Integrantes

Santiago Guerra Vásquez
Emmanuel [Apellido]
Jacobó [Apellido]

Septiembre de 2026

Introducción

Este documento reúne la estrategia que seguimos para resolver las dos preguntas del parcial. En los dos casos decidimos no partir de cero: usamos la ALU.hdl de 16 bits que ya habíamos construido y probado durante el curso como punto de partida, y desde ahí extendimos el diseño para cada problema. Nos pareció la manera más segura de trabajar, porque cualquier error nuevo que metiéramos iba a estar en la parte que agregábamos, no en la aritmética de base que ya sabíamos que funcionaba.

Las dos secciones siguen la misma estructura: primero explicamos la estrategia y las decisiones de diseño, después mostramos el diagrama correspondiente, y cerramos con cómo probamos que el chip funciona. La segunda pregunta agrega además el análisis de qué tan práctico es este tipo de diseño en un procesador real.

Pregunta 1 — ALU de 32 bits

1.1 Estrategia general

La idea central de ALU32 es partir cada operando de 32 bits en dos mitades de 16 bits — xLow/xHigh para X, yLow/yHigh para Y — y aplicarles la misma lógica de la ALU del curso a las dos mitades en paralelo, con los mismos seis bits de control (zx, nx, zy, ny, f, no) para ambas mitades a la vez. No creamos un segundo juego de bits de control para la mitad alta, porque el enunciado pide una ALU de 32 bits: una sola operación sobre un operando más ancho, no dos operaciones independientes de 16 bits compartiendo la misma instrucción.

1.2 Por qué el AND no necesita nada especial y la suma sí

Poner una mitad en cero o negarla no depende de la otra mitad: NOT de 32 bits es exactamente NOT de la mitad baja seguido de NOT de la mitad alta, y lo mismo pasa con anularla. Por eso los pasos 1 y 2 del HDL (procesar X y procesar Y) se hacen mitad por mitad sin ningún problema, igual que el AND del paso 3: cada bit del resultado depende solo del bit correspondiente de X y de Y, así que no hay ninguna interacción entre Low y High.

La suma es distinta. Si xLow + yLow se pasa de 16 bits, ese desborde tiene que sumarse a xHigh + yHigh, porque si no el resultado de 32 bits queda mal. Ese es el problema nuevo que no existe en la ALU de 16 bits del curso: hay que calcular y propagar un acarreo entre las dos mitades.

1.3 Cómo se detecta el acarreo entre mitades

El problema es que Add16, tal como está definido en Nand2Tetris, no expone ningún acarreo de salida — solo entrega el resultado de 16 bits. Para reconstruirlo usamos la fórmula clásica de generar/propagar un acarreo, a partir de los bits más significativos de los dos operandos y del bit más significativo de la suma:

```
generateCarry = x2Low[15] AND y2Low[15]
propagateCarry = x2Low[15] OR y2Low[15]
carryFromLow = generateCarry OR (propagateCarry AND NOT addLow[15])
```

La lógica es: si los dos bits 15 son 1, seguro hay acarreo (se generó uno nuevo). Si solo uno de los dos es 1, hay acarreo únicamente si la suma en esa posición dio 0, es decir, si se propagó un acarreo que venía de más abajo. Con carryFromLow ya armado, lo metemos como un bit extra en la suma de la mitad alta: $Add16(a=addHighBase, b[0]=carryFromLow, out=addHigh)$.

1.4 Diagrama de bloques

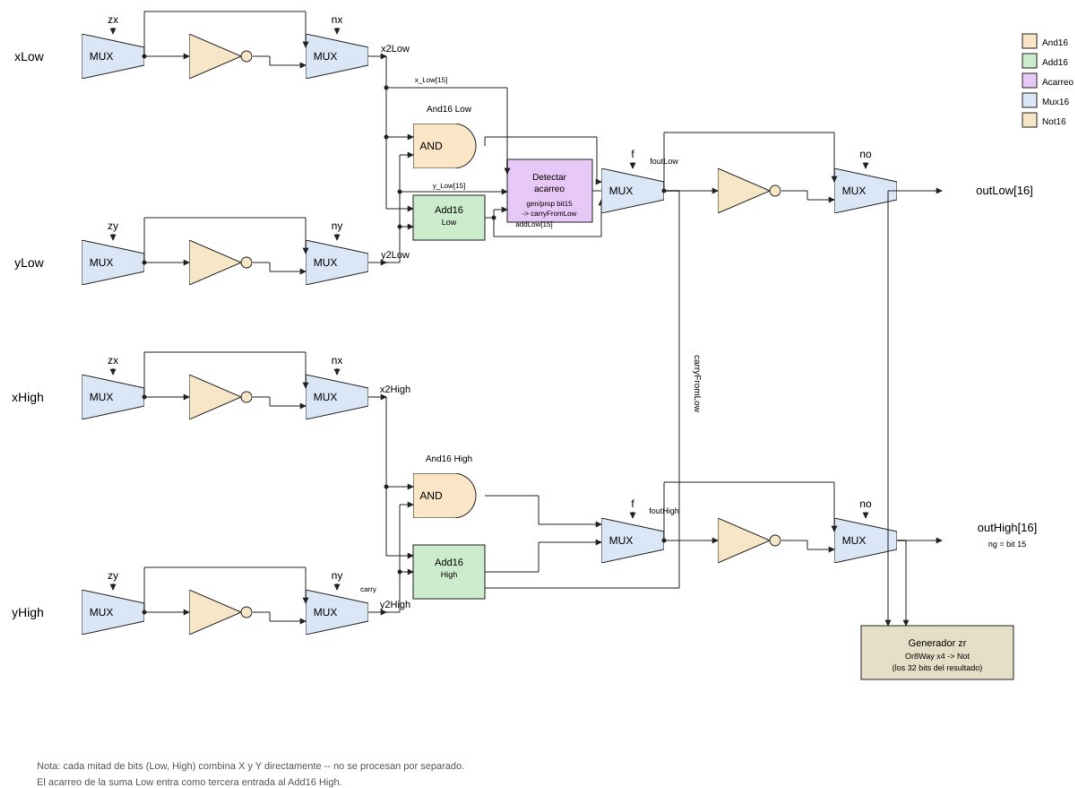


Figura 1. Diagrama de ALU32, organizado por mitad de bits (Low / High).

En el primer boceto habíamos agrupado las compuertas por operando: todo lo de X en un bloque arriba, todo lo de Y en un bloque abajo. Al revisarlo contra el HDL nos dimos cuenta de que eso daba a entender que el And16 y el Add16 procesaban xLow y xHigh juntos, y no es así: cada And16 y cada Add16 combina la misma mitad de X con la misma mitad de Y (x_{2Low} con y_{2Low} , x_{2High} con y_{2High}), nunca mezcla mitades entre sí. Por eso el diagrama final quedó organizado por mitad de bits (Low arriba, High abajo) en vez de por operando, con el bloque de detección de acarreo puesto exactamente donde se conecta la mitad baja con la alta.

1.5 Salida final: no, zr y ng

El bit no funciona igual que en la ALU de 16 bits, aplicado a cada mitad de la salida por separado. La bandera ng sale prácticamente gratis: como se trabaja en complemento a dos, el bit de signo de un número de 32 bits es el bit más alto de la mitad High, así que ng es directamente ese bit. La bandera zr sí pide más trabajo, porque hay que revisar los 32 bits del resultado completo: partimos la salida en cuatro bloques de 8 bits,

usamos Or8Way en cada uno para saber si tiene algún 1, y combinamos esas cuatro respuestas con OR — si ninguna tiene un 1, el resultado completo es cero.

1.6 Validación

Para probar ALU32 armamos ALU32.tst con 144 casos: las 18 funciones clásicas de la ALU de 16 bits aplicadas a 8 pares de X,Y distintos, elegidos a propósito para forzar casos donde el acarreo cruza de la mitad baja a la alta (por ejemplo $0x0000FFFF + 1$) y casos con el bit de signo en el límite ($0x7FFFFFFF + 1$). Los valores esperados no salieron de correr el simulador y confiar ciegamente en el resultado: los calculamos aparte en Python, replicando la aritmética de 32 bits bit a bit, y después comparamos ese cálculo contra la salida real del Hardware Simulator. Los 144 casos coincidieron exactamente; el archivo ALU32.cmp que queda en la entrega es justamente esa comparación (Comparison ended successfully).

1.7 Código fuente — ALU32.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/ALU32.hdl
/**
 * ALU32 (32-bit Arithmetic Logic Unit para Nand2Tetris):
 *
 * Entradas:
 *   xLow[16], xHigh[16] : Operando X (32 bits divididos en dos de 16)
 *   yLow[16], yHigh[16] : Operando Y (32 bits divididos en dos de 16)
 *   zx, nx, zy, ny, f, no : Bits de control
 *
 * Salidas:
 *   outLow[16], outHigh[16] : Resultado (32 bits divididos en dos de 16)
 *   zr : 1 si el resultado completo de 32 bits es 0, de lo contrario 0
 *   ng : 1 si el resultado es negativo (bit 15 de outHigh == 1)
 */

CHIP ALU32 {
    IN
        xLow[16], xHigh[16],
        yLow[16], yHigh[16],
        zx, // zero the x input?
        nx, // negate the x input?
        zy, // zero the y input?
        ny, // negate the y input?
        f, // compute (out = x + y) or (out = x & y)?
        no; // negate the out output?
    OUT
        outLow[16],
        outHigh[16],
        zr, // if (out == 0) equals 1, else 0
        ng; // if (out < 0) equals 1, else 0

    PARTS:

        // =====
        // PASO 1: Procesar entrada X (zx, nx)
        // =====
        Mux16(a=xLow, b=false, sel=zx, out=x1Low);
        Not16(in=x1Low, out=notx1Low);
        Mux16(a=x1Low, b=notx1Low, sel=nx, out=x2Low, out[15]=x2Low15);

        Mux16(a=xHigh, b=false, sel=zx, out=x1High);
```

```

Not16(in=x1High, out=notx1High);
Mux16(a=x1High, b=notx1High, sel=nx, out=x2High);

// =====
// PASO 2: Procesar entrada Y (zy, ny)
// =====
Mux16(a=yLow, b=false, sel=zy, out=y1Low);
Not16(in=y1Low, out=noty1Low);
Mux16(a=y1Low, b=noty1Low, sel=ny, out=y2Low, out[15]=y2Low15);

Mux16(a=yHigh, b=false, sel=zy, out=y1High);
Not16(in=y1High, out=noty1High);
Mux16(a=y1High, b=noty1High, sel=ny, out=y2High);

// =====
// PASO 3: Operación AND (f=0)
// =====
And16(a=x2Low, b=y2Low, out=andLow);
And16(a=x2High, b=y2High, out=andHigh);

// =====
// PASO 4: Operación ADD (f=1) con propagación de acarreo
// =====
// Suma mitad baja
Add16(a=x2Low, b=y2Low, out=addLow, out[15]=addLow15);

// Detección lógica del carry de la mitad baja:
// carry = (a15 & b15) | ((a15 | b15) & !sum15)
And(a=x2Low15, b=y2Low15, out=generateCarry);
Or(a=x2Low15, b=y2Low15, out=propagateCarry);
Not(in=addLow15, out=notAddLow15);
And(a=propagateCarry, b=notAddLow15, out=propagatedCarry);
Or(a=generateCarry, b=propagatedCarry, out=carryFromLow);

// Suma mitad alta + acarreo
Add16(a=x2High, b=y2High, out=addHighBase);
Add16(a=addHighBase, b[0]=carryFromLow, out=addHigh);

// =====
// PASO 5: Seleccionar función (f=0: AND, f=1: ADD)
// =====
Mux16(a=andLow, b=addLow, sel=f, out=foutLow);
Mux16(a=andHigh, b=addHigh, sel=f, out=foutHigh);

// =====
// PASO 6: Post-procesamiento (no) y generación de salidas
// =====
Not16(in=foutLow, out=notfoutLow);
Mux16(a=foutLow, b=notfoutLow, sel=no,
      out=outLow, out[0..7]=outByte0, out[8..15]=outByte1);

Not16(in=foutHigh, out=notfoutHigh);
Mux16(a=foutHigh, b=notfoutHigh, sel=no,
      out=outHigh, out[0..7]=outByte2, out[8..15]=outByte3, out[15]=ng);

// =====
// PASO 7: Flag zr (cero global de 32 bits)
// =====
Or8Way(in=outByte0, out=z0);
Or8Way(in=outByte1, out=z1);
Or8Way(in=outByte2, out=z2);
Or8Way(in=outByte3, out=z3);
Or(a=z0, b=z1, out=z01);
Or(a=z2, b=z3, out=z23);
Or(a=z01, b=z23, out=nonzero);
Not(in=nonzero, out=zr);

```

}

Pregunta 2 — ALU de tres entradas

2.1 Estrategia general

Para la ALU3 la decisión de diseño fue distinta a la de ALU32. En vez de armar una red de compuertas nueva que reciba tres operandos a la vez, usamos dos veces la ALU de 16 bits que ya existía en el curso (ALU.hdl), una después de la otra: la primera combina X e Y y produce un resultado intermedio w; la segunda combina w con Z. Cada etapa usa el mismo patrón de control que ya conocíamos (zx, nx, zy, ny, f, no), solo que la segunda etapa tiene su propio juego de cuatro bits (zz, nz, fz, noz) para decidir qué le hace a Z y qué función aplica entre w y Z.

2.2 Por qué w entra sin zx/nx propios

La entrada w de la segunda ALU se conecta con zx=false y nx=false fijos, en vez de darle su propio par de bits. La razón es que w ya sale completamente procesado de la primera etapa: ponerlo en cero o negarlo otra vez ahí no agrega ninguna operación que no se pudiera lograr ya ajustando los bits de la etapa 1. A cambio de esa simplificación, la ALU3 queda con 10 bits de control en total en vez de 12, aunque eso también significa que hay funciones de la ALU clásica que no se pueden pedir de forma aislada sobre w en la segunda etapa — por ejemplo, no se puede pedir !w solo, sin combinarlo con Z; hay que lograrlo componiéndolo dentro de la etapa 1.

2.3 Conjunto de operaciones y bits de control

Con 6 bits para la etapa 1 y 4 para la etapa 2 hay $2^{10} = 1024$ combinaciones posibles. Para este documento elegimos las que nos parecieron más representativas de lo que puede hacer una ALU de tres entradas en la práctica:

Operación	zx nx zy ny f no	zz nz fz noz
$x + y + z$	0 0 0 0 1 0	0 0 1 0
$(x \text{ AND } y) + z$	0 0 0 0 0 0	0 0 1 0
$(x + y) \text{ AND } z$	0 0 0 0 1 0	0 0 0 0
$x + y - z$	0 0 0 0 1 0	0 1 1 0
$x - y + z$	0 1 0 0 1 1	0 0 1 0
$x + y$ (z ignorado)	0 0 0 0 1 0	1 0 1 0
$x + y + 1$	0 0 0 0 1 0	1 1 1 1
$-(x + y + z)$	0 0 0 0 1 0	0 0 1 1
$(x + y) - 1$	0 0 0 0 1 0	1 1 1 0

2.4 Diagrama de circuito

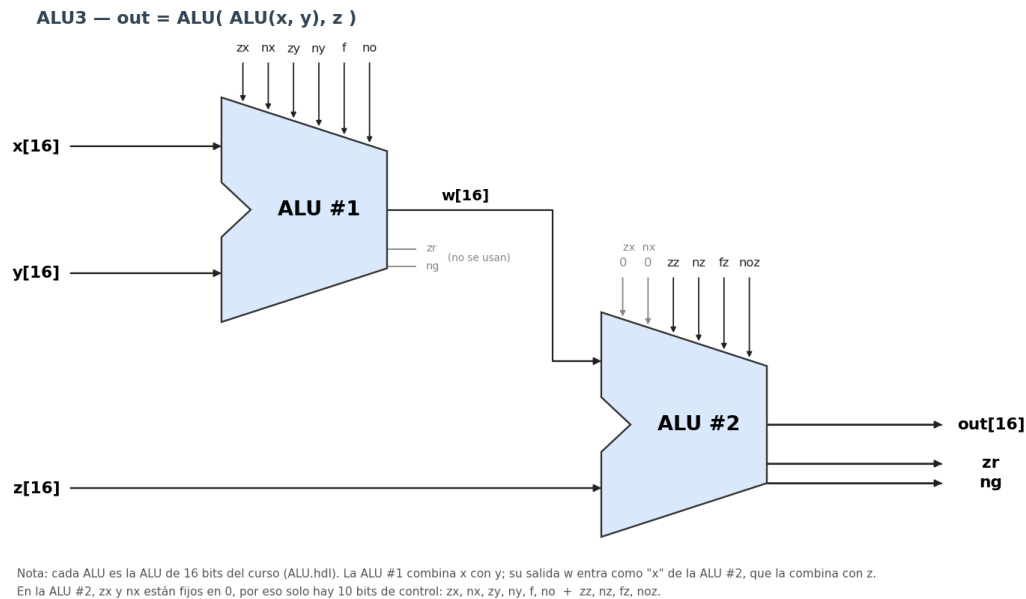


Figura 2. ALU3 como dos instancias del chip ALU encadenadas.

A diferencia del diagrama de ALU32, acá no hace falta bajar al nivel de compuertas individuales, porque el punto del diseño es justo el contrario: no se agregó ninguna compuerta nueva. El diagrama muestra las dos instancias de ALU como bloques, con X y Y entrando a la primera, w saliendo de ahí hacia la segunda junto con Z, y out, zr y ng saliendo de la segunda.

2.5 Validación

El archivo ALU3.tst tiene 42 casos: cruzamos 6 funciones representativas de la etapa 1 (x+y, x-y, x AND y, -x, x+1, y la función que da 0) con 6 de la etapa 2 (w+z, w-z, w AND z, z anulado, -(w+z), y z ignorado), probados sobre 7 combinaciones de x, y, z que incluyen los tres operandos en cero, los tres en -1 (0xFFFF), overflow con signo al encadenar dos sumas (32767+1+1) y el valor más negativo de 16 bits sumado a operandos chicos. Igual que con ALU32, cada valor esperado se calculó aparte en Python antes de correr el simulador, y los 42 coincidieron (Comparison ended successfully).

2.6 Análisis de arquitectura: ¿tiene sentido fabricar esto de verdad?

Nuestra respuesta corta es que no, al menos no como instrucción de propósito general de un solo ciclo, aunque el ejercicio sí deja ver en qué contexto sí tendría sentido.

El primer problema aparece en el set de instrucciones. Una ALU de dos entradas se controla con 6 bits, que caben sin apretar en el opcode de una instrucción de 16 bits como las de Hack. La ALU3 necesita 10 bits de control más una tercera dirección de registro fuente (X, Y y Z, en vez de solo X e Y). Eso ya no entra en una instrucción de 16 bits sin quitarle espacio a la parte que direcciona memoria o registros, así que en la práctica obligaría a usar instrucciones más largas (32 bits) o a partir la operación en dos palabras — una de control y otra de operandos —, lo que complica bastante el decodificador de instrucciones.

El segundo problema es el consumo de energía. El camino de datos de la ALU3 son dos ALUs completas de 16 bits en serie, así que cada operación mueve aproximadamente el doble de compuertas que una ALU normal de dos entradas. Eso pasa incluso cuando Z no hace falta ($zz=1$): la segunda ALU igual procesa $w+0$ puerta por puerta, porque el hardware no tiene forma de saltarse la etapa si no se necesita.

El tercer problema es la unidad de control y el camino crítico. No solo hay que decodificar 10 señales en vez de 6; la salida w de la primera etapa es la entrada de la segunda, así que la señal tiene que atravesar dos sumadores completos en cascada antes de estabilizarse. Eso alarga el tiempo de propagación de la ALU entera, y en un diseño síncrono normal eso se traduce en bajar la frecuencia de reloj o partir la operación en dos ciclos con un registro guardando w a la mitad, lo que a su vez pide más lógica de control.

Juntando los tres puntos, no nos parece que valga la pena como instrucción genérica: el costo en área, energía y camino crítico no se justifica frente a simplemente ejecutar dos instrucciones normales de la ALU de dos entradas seguidas ($w = x \text{ op } y$, y después $\text{out} = w \text{ op } z$), que es literalmente lo que la ALU3 hace por dentro, solo que fusionado en una sola instrucción. Donde sí le vemos sentido es como instrucción fusionada especializada — al estilo de un fused multiply-add — dentro de una unidad vectorial o un DSP, donde combinar tres operandos es tan frecuente (acumuladores, filtros FIR) que amortizar ese costo extra en una unidad dedicada, fuera del pipeline de propósito general, sí compensa.

2.7 Código fuente — ALU3.hdl

```
// File name: projects/2/ALU3.hdl
/**
 * ALU3 (ALU de 3 entradas para Nand2Tetris):
 *
 * Calcula out = fz( no( f( zx?0:x , zy?0:y ) ) , zz?0:z )
 * es decir: primero combina X e Y exactamente como la ALU clásica de 16 bits
 * (usando los 6 bits de control conocidos: zx, nx, zy, ny, f, no), y el
 * resultado de esa combinación (w) se vuelve a combinar con Z usando el
 * mismo patrón (zz, nz, fz, noz).
 *
 * Entradas:
 *   x[16], y[16], z[16] : Operandos, 16 bits cada uno
 *   zx, nx, zy, ny, f, no : Bits de control de la etapa X-Y (igual que ALU.hdl)
 *   zz, nz, fz, noz      : Bits de control de la etapa (X op Y) - Z
 *
 * Salidas:
 *   out[16] : Resultado final
 *   zr : 1 si out == 0
 *   ng : 1 si out < 0
 */

CHIP ALU3 {
    IN  x[16], y[16], z[16],
        zx, nx, zy, ny, f, no,
        zz, nz, fz, noz;
    OUT out[16], zr, ng;

    PARTS:
        // Etapa 1: combina X e Y exactamente como la ALU clasica de 16 bits
        ALU(x=x, y=y, zx=zx, nx=nx, zy=zy, ny=ny, f=f, no=no, out=w);

        // Etapa 2: combina el resultado w con Z usando el mismo patron
        // (w hace de "x", con zx=nx=false porque w ya viene completamente
        // procesado por la etapa 1 y no necesita un segundo cero/negacion)
```

```
    ALU(x=w, y=z, zx=false, nx=false, zy=zz, ny=nz, f=fz, no=noz, out=out, zr=zr, ng=ng);  
}
```

Cierre

El código HDL completo de las dos ALUs, los scripts de prueba (.tst / .cmp) y los diagramas de esta entrega están en la carpeta "entrega 1 OC" del repositorio del equipo. Al momento de escribir este documento, el video de sustentación de ambas preguntas sigue en proceso de grabación.